MySQL para Iniciantes: do Zero ao Avançado

Este guia foi escrito para quem **nunca mexeu com banco de dados**. Ele não exige nenhum conhecimento prévio: toda palavra técnica é explicada na primeira vez que aparece, e há um glossário no final para consultar quando bater a dúvida.

> **Como estudar:** leia uma seção por vez e digite os exemplos você mesmo, em vez de copiar e colar. Digitar fixa muito mais. Errar faz parte: num banco de testes, o pior que pode acontecer é você apagar tudo e começar de novo.

Etapa	Seções	O que você aprende
Fundamentos	1 e 2	O que é um banco de dados e como deixar tudo pronto no seu computador
Primeiros comandos	3 a 5	Criar tabelas, guardar dados e buscar o que você precisa
Análise de dados	6 a 8	Contar, somar, agrupar e cruzar informações de várias tabelas
Nível avançado	9 e 10	Deixar consultas rápidas, proteger os dados e fazer backup
Prática	11	Exercícios com respostas, erros comuns e glossário

Sumário: clique para ir direto à seção

Seção	Em uma frase
1. O Básico do Básico	O que são banco de dados, SQL e MySQL
2. Preparando o Ambiente	Instalar e rodar o primeiro comando
3. Criando seu Banco e sua Primeira Tabela	`CREATE`, tipos de dados e regras das colunas
4. Guardando, Alterando e Apagando Dados	`INSERT`, `UPDATE` e `DELETE`
5. Fazendo Perguntas ao Banco: SELECT	Filtros, ordem, limites e o `NULL`
6. Resumindo Dados	Contar, somar, agrupar, textos e datas
7. Mais de uma Tabela: Relacionamentos e JOIN	Chaves, `JOIN` e o mapa do banco

Seção	Em uma frase
8. Consultas Dentro de Consultas	Subconsultas, CTEs e `UNION`
9. Recursos Avançados, com Calma	Índices, transações, rankings e views
10. Segurança e Backup	Usuários, SQL injection e cópias de segurança
11. Praticando: Exercícios, Erros Comuns e Glossário	Exercícios com respostas, glossário e plano de estudo

1. O Básico do Básico

Antes de qualquer comando, vamos entender as peças do quebra-cabeça. Esta seção não tem código nenhum para digitar: é só a base para tudo que vem depois.

O que é um banco de dados?

Um **banco de dados** é um lugar organizado para guardar informações, de um jeito que seja fácil **achar, alterar e combinar** essas informações depois. Pense em um fichário de escritório, em uma agenda de contatos ou em uma planilha do Excel muito bem arrumada.

Você usa bancos de dados o dia todo sem perceber:

- O Instagram guarda seus posts, curtidas e seguidores.
- Uma loja online guarda produtos, clientes e pedidos.
- O aplicativo do seu banco guarda seu saldo e seu extrato.
- O WhatsApp guarda o histórico das conversas.

Tabelas, linhas e colunas

Um banco de dados é dividido em **tabelas**. Cada tabela guarda um assunto: uma tabela para clientes, outra para produtos, outra para pedidos. Se você já viu uma planilha, já conhece o formato:

id	nome	email	cidade
1	Ana	ana@email.com	Rio de Janeiro
2	Bruno	bruno@email.com	Niterói
3	Carla	carla@email.com	São Paulo

Essa é a tabela `clientes`. Nela:

Termo	O que é	Na tabela acima
Tabela	Um conjunto de informações sobre um assunto	A tabela inteira, chamada `clientes`
Coluna (ou campo)	Um tipo de informação, igual para todas as linhas	`nome`, `email`, `cidade`
Linha (ou registro)	Uma ocorrência completa, um "item"	A linha da Ana
Chave primária	Uma coluna que identifica cada linha de forma única, como o CPF de uma pessoa	A coluna `id`

> **Por que ter um `id`?** Dois clientes podem se chamar "Ana". O `id` é o número único que diferencia uma da outra, sem confusão.

Por que não usar só o Excel?

Para pouca informação, a planilha resolve. Mas quando o volume e a complexidade crescem, o banco de dados leva vantagem:

Situação	Planilha	Banco de dados
Milhões de linhas	Fica lenta ou trava	Feito para isso
Várias pessoas editando ao mesmo tempo	Conflitos e versões duplicadas	Controla tudo sem bagunça
Evitar erros (e-mail repetido, preço negativo)	Depende de cuidado manual	Você cria regras e ele aplica sozinho
Ligar informações (cliente e seus pedidos)	Fórmulas complicadas	É o ponto forte
Sites e aplicativos usarem os dados	Difícil	Programas conversam direto com ele

O que é SQL?

SQL (Structured Query Language, ou "linguagem de consulta estruturada") é a **linguagem usada para conversar com o banco de dados**. Lê-se "ésse-quiu-éli" ou "síquel". Uma **consulta** (em inglês, *query*) é uma pergunta ou ordem que você faz ao banco.

É como pedir algo ao garçom: você diz o que quer, e ele cuida do como. Os comandos parecem inglês simples:

```
SELECT nome FROM clientes WHERE cidade = 'Rio de Janeiro';
```

Tradução em português: *"Selecione o nome, da tabela de clientes, onde a cidade for Rio de Janeiro."* O banco responde com uma lista de nomes.

O que é MySQL?

SQL é a **linguagem**. **MySQL** é o **programa** que guarda os dados e entende essa linguagem. Esse tipo de programa se chama **SGBD** (Sistema Gerenciador de Banco de Dados).

Uma analogia: SQL é o idioma, e o MySQL é o funcionário do arquivo que fala esse idioma e cuida das gavetas para você.

Existem outros programas parecidos: PostgreSQL, SQL Server, Oracle e SQLite. Todos falam SQL, com pequenas diferenças de dialeto, então o que você aprende aqui vale para os outros também. O MySQL é um dos mais populares do mundo e roda por trás de sites como WordPress e lojas WooCommerce.

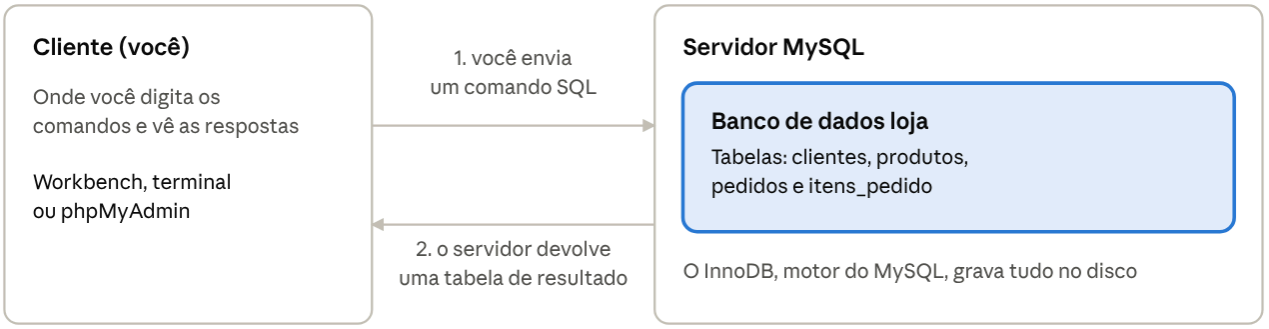
Cliente e servidor

Você vai ouvir essas duas palavras o tempo todo:

Palavra	O que é	Analogia do restaurante
Servidor	O programa MySQL, que fica ligado guardando os dados e respondendo pedidos	A cozinha
Cliente	O programa onde você digita seus comandos e vê as respostas (Workbench, terminal, phpMyAdmin)	Você na mesa, fazendo o pedido

No dia a dia, os dois podem estar no mesmo computador (o seu) ou o servidor pode ficar em outra máquina, na internet.

Você pergunta em SQL e o servidor MySQL responde com uma tabela



cliente e servidor · 2 setas

Cada comando que você digita viaja até o servidor, que consulta o banco e devolve o resultado em forma de tabela.

Os comandos que você mais vai usar

Quase todo o trabalho com bancos de dados se resume a cinco ações:

Ação	Comando	Exemplo em português
Criar a estrutura	`CREATE`	"Crie uma tabela de clientes"
Guardar um dado	`INSERT`	"Adicione a cliente Ana"
Consultar	`SELECT`	"Mostre todos os clientes do Rio"
Alterar	`UPDATE`	"Mude a cidade da Ana para São Paulo"
Apagar	`DELETE`	"Remova o cliente 3"

> **E aquele tal de InnoDB?** É o "motor de armazenamento" do MySQL: a peça interna que de fato grava e organiza os dados no disco. O InnoDB é o motor padrão e o recomendado, e **você não precisa configurar nada**. Basta saber que é ele quem permite recursos como transações e ligações entre tabelas, que veremos mais à frente.

2. Preparando o Ambiente

Para praticar, você precisa de um MySQL funcionando. Há dois caminhos: um rápido, sem instalar nada, e outro completo, instalando no seu computador.

Caminho A: sem instalar nada (para começar hoje)

Existem sites que oferecem um MySQL pronto no navegador, onde você digita os comandos e vê o resultado. Pesquise por **"MySQL online"** e procure serviços como **DB Fiddle** ou **OneCompiler**. É ótimo para as primeiras seções. A desvantagem: o que você cria pode se perder ao fechar a página.

Caminho B: instalando no seu computador (recomendado)

Você vai instalar dois programas: o **servidor** MySQL (quem guarda os dados) e o **MySQL Workbench** (o cliente, uma janela onde você digita comandos e vê as tabelas). Ambos são gratuitos e baixados do site oficial, `dev.mysql.com/downloads`.

No Windows:

1. Baixe o **MySQL Installer** e abra.
2. Escolha o tipo de instalação **Developer Default**. Ele instala o servidor e o Workbench juntos.
3. Siga o assistente com "Next" até pedir uma **senha para o usuário `root`**. O `root` é o administrador do MySQL, o "dono da casa". Crie uma senha e **anote em um lugar seguro**.
4. Termine a instalação e abra o **MySQL Workbench**.
5. Na tela inicial, clique na conexão chamada **Local instance** e digite a senha do `root`.

No Mac: baixe o instalador `.dmg` no mesmo site e siga os passos, ou use o Homebrew (`brew install mysql`).

No Linux (Ubuntu ou Debian): abra o terminal e digite:

```
sudo apt update
sudo apt install mysql-server
sudo mysql_secure_installation
```

> **O que é o terminal?** É uma janela de texto onde você controla o computador digitando comandos, em vez de clicar. Se preferir clicar, use o Workbench, que funciona em qualquer sistema.

Seu primeiro comando

No Workbench, abra uma aba de consulta (o botão com o ícone de papel e a sigla **SQL**, no topo). Digite:

```
SELECT 'Olá, mundo!';
```

Para **executar**, clique no ícone de **raio** ou aperte `Ctrl + Enter` (no Mac, `Cmd + Enter`). Embaixo, aparece uma tabela de resultado com o texto. Parabéns, você acaba de conversar com um banco de dados.

Tente também:

```
SELECT 2 + 3;        -- o MySQL também faz contas: resultado 5
SELECT NOW();        -- data e hora de agora
SHOW DATABASES;      -- lista os bancos que já existem
```

Regras de escrita (leia uma vez e pronto)

Regra	Detalhe
Ponto e vírgula `;`	Marca o **fim de cada comando**. Esqueceu, o MySQL fica esperando mais texto
Maiúsculas e minúsculas	Os comandos (`SELECT`, `FROM`) funcionam de qualquer forma. Por convenção, escrevemos os comandos em MAIÚSCULAS e os nomes em minúsculas, para ler melhor
Texto entre aspas simples	Textos vão entre aspas simples: `'Ana'`. Números não levam aspas: `10`
Comentários	Tudo depois de `--` (com um espaço depois) é ignorado. Serve para anotações. Também vale `# texto`
Várias linhas	Pode quebrar o comando em quantas linhas quiser; só o `;` encerra
Nomes	Use letras minúsculas, sem acento e sem espaço. Para separar palavras, use `_`: `data_nascimento`

Se algo der errado

Mensagem de erro	O que costuma ser
`Access denied for user 'root'`	Senha errada. Confira a senha que você criou na instalação
`Can't connect to MySQL server`	O servidor não está ligado. No Windows, procure "Serviços" e inicie o MySQL

Mensagem de erro	O que costuma ser
`You have an error in your SQL syntax`	Erro de digitação: falta de vírgula, aspas ou parêntese. A mensagem indica perto de onde
`Unknown column` / `Table doesn't exist`	Nome escrito errado, ou o banco certo não está selecionado

> **Dica:** ler a mensagem de erro com calma resolve 80% dos problemas. Ela quase sempre aponta a linha e a palavra onde o MySQL se perdeu.

3. Criando seu Banco e sua Primeira Tabela

Vamos construir, passo a passo, o banco de dados de uma **loja**. Ele vai acompanhar você até o fim do guia.

Passo 1: criar o banco

Um servidor MySQL pode guardar vários bancos, como um prédio com vários apartamentos. Cada projeto costuma ter o seu.

```
CREATE DATABASE loja CHARACTER SET utf8mb4;
USE loja;
```

- `CREATE DATABASE loja` cria o banco chamado `loja`.
- `CHARACTER SET utf8mb4` diz como os textos são gravados. Esse valor aceita **acentos, cedilha e emojis** sem problemas. Use sempre.
- `USE loja` significa "a partir de agora, trabalhe dentro do banco `loja`". Sem isso, o MySQL não sabe em qual banco você quer mexer.

Passo 2: entender os tipos de dados

Ao criar uma coluna, você precisa dizer **que tipo de informação** ela guarda. Isso ajuda o MySQL a economizar espaço e a recusar valores sem sentido (por exemplo, texto onde deveria haver um número).

Tipo	Para guardar	Exemplo
`INT`	Números inteiros	`1`, `42`, `-7`

Tipo	Para guardar	Exemplo
`DECIMAL(10,2)`	Números com vírgula e precisão exata, ideais para **dinheiro**. `10` é o total de dígitos e `2` os dígitos depois da vírgula	`1999.90`
`VARCHAR(100)`	Texto curto, de até 100 caracteres	`'Ana Souza'`
`TEXT`	Texto longo (descrições, comentários)	Um parágrafo inteiro
`DATE`	Uma data, no formato ano-mês-dia	`'2026-10-08'`
`DATETIME`	Data e hora juntas	`'2026-10-08 14:30:00'`
`BOOLEAN`	Sim ou não (guarda `1` ou `0`)	`TRUE`, `FALSE`

> **Importante:** para valores em dinheiro, use sempre `DECIMAL`. O tipo `FLOAT` faz pequenos arredondamentos que, somados, geram centavos errados.

Passo 3: criar a tabela de clientes

```
CREATE TABLE clientes (
  id        INT AUTO_INCREMENT PRIMARY KEY,
  nome      VARCHAR(100) NOT NULL,
  email     VARCHAR(120) NOT NULL UNIQUE,
  cidade    VARCHAR(60),
  criado_em DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Vamos ler linha por linha:

Trecho	Significado
`CREATE TABLE clientes (`	Crie uma tabela chamada `clientes`. As colunas ficam entre parênteses
`id INT AUTO_INCREMENT PRIMARY KEY`	Coluna `id`, número inteiro, que o MySQL **numera sozinho** (1, 2, 3...) e que identifica cada linha de forma única
`nome VARCHAR(100) NOT NULL`	Texto de até 100 caracteres, **obrigatório** (não pode ficar vazio)
`email VARCHAR(120) NOT NULL UNIQUE`	Obrigatório e **único**: não aceita dois clientes com o mesmo e-mail

Trecho	Significado
`cidade VARCHAR(60)`	Texto opcional (sem `NOT NULL`, pode ficar vazio)
`criado_em DATETIME DEFAULT CURRENT_TIMESTAMP`	Se você não informar, o MySQL preenche com a data e hora do momento do cadastro

Cada coluna é separada da próxima por **vírgula**, e a última não leva vírgula.

Passo 4: criar a tabela de produtos

```
CREATE TABLE produtos (
  id        INT AUTO_INCREMENT PRIMARY KEY,
  nome      VARCHAR(100) NOT NULL,
  categoria VARCHAR(50),
  preco     DECIMAL(10,2) NOT NULL
);
```

As "regras" das colunas (constraints)

Essas palavras que você viu se chamam **restrições** (em inglês, *constraints*). São regras que o banco faz cumprir por você:

Restrição	O que garante
`PRIMARY KEY`	Cada linha tem um identificador único e nunca vazio
`AUTO_INCREMENT`	O número do `id` é gerado automaticamente
`NOT NULL`	A coluna é obrigatória
`UNIQUE`	Não pode haver valores repetidos na coluna
`DEFAULT`	Valor automático quando você não informa nenhum

Conferindo o que você criou

```
SHOW TABLES;          -- lista as tabelas do banco
DESCRIBE clientes;    -- mostra as colunas e seus tipos
```

Mudando uma tabela depois de criada

Esqueceu uma coluna? Dá para mexer sem recriar tudo, com `ALTER TABLE` ("alterar tabela"):

```
ALTER TABLE clientes ADD COLUMN telefone VARCHAR(20);    -- adiciona coluna
ALTER TABLE clientes DROP COLUMN telefone;               -- remove a coluna
```

Para apagar uma tabela inteira, use `DROP TABLE nome;`.

> **Cuidado:** `DROP` apaga a tabela **e todos os dados dela, sem volta e sem pedir confirmação**. Em um banco de testes tudo bem; em um banco real, pense duas vezes e tenha backup.

4. Guardando, Alterando e Apagando Dados

As tabelas existem, mas estão vazias, como planilhas sem nenhuma linha. Vamos preenchê-las.

INSERT: adicionar dados

`INSERT INTO` significa "inserir em". Você diz em qual tabela, quais colunas e quais valores.

```
INSERT INTO clientes (nome, email, cidade)
VALUES ('Ana Souza', 'ana@email.com', 'Rio de Janeiro');
```

Leitura: *"Insira na tabela `clientes`, nas colunas `nome`, `email` e `cidade`, os valores `Ana Souza`, `ana@email.com` e `Rio de Janeiro`."* Repare que **a ordem dos valores segue a ordem das colunas**. Não informamos `id` (o MySQL numera sozinho) nem `criado_em` (usa a data de agora).

Para adicionar várias linhas de uma vez, separe-as por vírgula:

```
INSERT INTO clientes (nome, email, cidade) VALUES
  ('Bruno Lima', 'bruno@email.com', 'Niterói'),
  ('Carla Dias', 'carla@email.com', 'São Paulo'),
  ('Diego Rocha', 'diego@email.com', NULL),
  ('Elisa Nunes', 'elisa@email.com', 'Rio de Janeiro');

INSERT INTO produtos (nome, categoria, preco) VALUES
  ('Caderno 20 matérias', 'Papelaria',   18.90),
  ('Caneta azul',         'Papelaria',    2.50),
  ('Mochila escolar',     'Acessórios', 129.90),
  ('Fone de ouvido',      'Eletrônicos', 89.90),
  ('Mouse sem fio',       'Eletrônicos', 59.90),
  ('Teclado',             'Eletrônicos', 149.90),
  ('Garrafa térmica',     'Acessórios',  45.00),
  ('Livro de SQL',        'Livros',      79.90);
```

O `NULL` do Diego significa **"não sei / não informado"**. É diferente de zero e de texto vazio; falaremos mais sobre isso na próxima seção.

Para ver o que ficou guardado, use `SELECT * FROM`, que significa "mostre tudo (`*`) da tabela":

```
SELECT * FROM clientes;
```

id	nome	email	cidade	criado_em
1	Ana Souza	ana@email.com	Rio de Janeiro	2026-10-08 14:30:00
2	Bruno Lima	bruno@email.com	Niterói	2026-10-08 14:30:00
3	Carla Dias	carla@email.com	São Paulo	2026-10-08 14:30:00
4	Diego Rocha	diego@email.com	NULL	2026-10-08 14:30:00
5	Elisa Nunes	elisa@email.com	Rio de Janeiro	2026-10-08 14:30:00

> **Testando uma regra:** tente inserir de novo um cliente com o e-mail `ana@email.com`. O MySQL vai recusar com o erro `Duplicate entry ... for key 'email'`, porque a coluna é `UNIQUE`. É a restrição trabalhando para você.

UPDATE: alterar dados

`UPDATE` significa "atualizar". Você diz qual tabela, o que mudar (`SET`) e **em quais linhas** (`WHERE`).

```
UPDATE clientes
SET cidade = 'São Paulo'
WHERE id = 2;
```

Leitura: *"Atualize a tabela `clientes`: coloque a cidade como São Paulo, onde o `id` for 2."* Só o Bruno muda.

Para alterar mais de uma coluna, separe por vírgula: `SET cidade = 'Niterói', nome = 'Bruno L. Lima'`.

DELETE: apagar dados

`DELETE FROM` significa "apagar de". Também precisa do `WHERE`:

```
DELETE FROM clientes
WHERE id = 4;
```

Leitura: *"Apague da tabela `clientes` a linha onde o `id` for 4."* O Diego sai da tabela.

O perigo do WHERE esquecido

O `WHERE` é o que escolhe **quais linhas** sofrem a ação. Sem ele, o comando vale para **todas as linhas da tabela**:

```
UPDATE clientes SET cidade = 'São Paulo';   -- TODOS os clientes agora moram
em São Paulo!
DELETE FROM clientes;                       -- apaga TODOS os clientes!
```

> **Regra de ouro:** antes de rodar um `UPDATE` ou `DELETE`, troque-o por um `SELECT` com o mesmo `WHERE` e confira quais linhas aparecem. Se forem as certas, aí sim rode o comando de verdade.

```
SELECT * FROM clientes WHERE id = 4;    -- confira antes
DELETE FROM clientes WHERE id = 4;      -- depois apague
```

O Workbench tem uma proteção chamada **Safe Updates**: ele bloqueia `UPDATE` e `DELETE` que não usam a chave primária no `WHERE`, com o erro 1175. Ela existe para evitar acidentes, então prefira filtrar pelo `id`.

Resumo da seção

Quero...	Comando	Esqueleto
Adicionar	`INSERT`	`INSERT INTO tabela (colunas) VALUES (valores);`
Alterar	`UPDATE`	`UPDATE tabela SET coluna = valor WHERE condição;`
Apagar	`DELETE`	`DELETE FROM tabela WHERE condição;`

5. Fazendo Perguntas ao Banco: SELECT

O `SELECT` ("selecionar") é o comando mais usado de todos. Com ele você **faz perguntas** ao banco e recebe respostas em forma de tabela. Ele não altera nada: é seguro para experimentar à vontade.

> **Voltando ao ponto de partida:** na seção anterior você mudou o Bruno para São Paulo e apagou o Diego. Para os resultados batarem com os desta seção, desfaça isso:
>
> `UPDATE clientes SET cidade = 'Niterói' WHERE id = 2;`
>
> `INSERT INTO clientes (id, nome, email) VALUES (4, 'Diego Rocha', 'diego@email.com');`

A estrutura básica

```
SELECT   colunas que quero ver
FROM     tabela onde estão os dados
WHERE    condição para filtrar as linhas   -- opcional
ORDER BY como ordenar                      -- opcional
LIMIT    quantas linhas no máximo;         -- opcional
```

Nessa ordem, sempre. Pense assim: **o que** eu quero ver, **de onde**, **com qual filtro**, **em que ordem**.

Escolhendo as colunas

```
SELECT nome, cidade FROM clientes;
```

nome	cidade
Ana Souza	Rio de Janeiro
Bruno Lima	Niterói
Carla Dias	São Paulo
Diego Rocha	NULL
Elisa Nunes	Rio de Janeiro

O `*` mostra todas as colunas (`SELECT * FROM clientes`). É prático para explorar, mas em sistemas reais prefira listar só as colunas necessárias: fica mais rápido e mais claro.

Apelidos e contas: com `AS` você dá um nome novo à coluna do resultado (o nome na tabela não muda), e pode fazer cálculos:

```
SELECT nome, preco * 0.9 AS preco_com_desconto FROM produtos WHERE id IN (1, 3);
```

nome	preco_com_desconto
Caderno 20 matérias	17.010
Mochila escolar	116.910

Filtrando linhas com WHERE

O `WHERE` ("onde") deixa passar só as linhas que cumprem a condição.

```
SELECT nome, preco FROM produtos WHERE preco > 100;
```

nome	preco
Mochila escolar	129.90
Teclado	149.90

Operadores de comparação:

Símbolo	Significa	Exemplo
`=`	igual a	`cidade = 'Niterói'`

Símbolo	Significa	Exemplo
`<>` ou `!=`	diferente de	`categoria <> 'Livros'`
`>` e `<`	maior que, menor que	`preco > 100`
`>=` e `<=`	maior ou igual, menor ou igual	`preco <= 50`

Para combinar condições, use `AND` (as duas precisam ser verdadeiras) e `OR` (basta uma):

```
SELECT nome, preco FROM produtos
WHERE categoria = 'Eletrônicos' AND preco < 100;
```

nome	preco
Fone de ouvido	89.90
Mouse sem fio	59.90

> **Use parênteses quando misturar `AND` e `OR`.** O `AND` é resolvido antes do `OR`, e isso causa resultados inesperados. Escreva `(a OR b) AND c` quando for essa a sua intenção.

Atalhos muito úteis

Atalho	Quando usar	Exemplo	Resultado
`IN (...)`	Valor está em uma lista	`categoria IN ('Papelaria', 'Livros')`	Caderno, Caneta, Livro de SQL
`BETWEEN a AND b`	Valor dentro de uma faixa, **incluindo** as pontas	`preco BETWEEN 40 AND 90`	Fone, Mouse, Garrafa, Livro
`LIKE`	Busca por parte do texto	`nome LIKE 'M%'`	Mochila escolar, Mouse sem fio

No `LIKE`, o `%` representa "qualquer coisa, de qualquer tamanho" e o `_` representa "exatamente um caractere":

- `'M%'`: começa com M.
- `'%sem%'`: contém "sem" em qualquer lugar.

- `'%fio'`: termina com "fio".

O MySQL **não diferencia maiúsculas de minúsculas** nessas comparações: `'m%'` acha o mesmo que `'M%'`.

O famoso NULL (o "vazio")

`NULL` quer dizer **"valor desconhecido ou não informado"**. Não é zero nem texto vazio. O Diego não tem cidade cadastrada, e não sabemos qual é.

A pegadinha: qualquer comparação com `NULL` com `=` dá "desconhecido", nunca verdadeiro. Por isso:

```
SELECT nome FROM clientes WHERE cidade = NULL;    -- ERRADO: nunca devolve
nada
SELECT nome FROM clientes WHERE cidade IS NULL;   -- CERTO: devolve o Diego
SELECT nome FROM clientes WHERE cidade IS NOT NULL;  -- quem tem cidade
```

Para trocar o vazio por um texto na hora de mostrar, use `IFNULL`:

```
SELECT nome, IFNULL(cidade, 'Não informada') AS cidade FROM clientes;
```

Ordenando com ORDER BY

```
SELECT nome, preco FROM produtos ORDER BY preco DESC;
```

`ASC` ordena do menor para o maior (é o padrão) e `DESC` do maior para o menor. Dá para ordenar por vários critérios: `ORDER BY categoria ASC, preco DESC`.

Limitando com LIMIT

```
SELECT nome, preco FROM produtos ORDER BY preco DESC LIMIT 3;
```

nome	preco
Teclado	149.90
Mochila escolar	129.90
Fone de ouvido	89.90

Isso responde "quais são os 3 produtos mais caros?". Para pular linhas (útil em páginas de um site), use `LIMIT 10 OFFSET 20`: "traga 10 linhas, depois de pular as 20 primeiras".

Removendo repetições com DISTINCT

```
SELECT DISTINCT cidade FROM clientes;
```

Mostra cada cidade uma vez só, em vez de repetir "Rio de Janeiro" para a Ana e para a Elisa.

Tudo junto

```
SELECT nome, categoria, preco
FROM produtos
WHERE categoria IN ('Eletrônicos', 'Acessórios')
  AND preco < 100
ORDER BY preco ASC
LIMIT 3;
```

Leitura: *"Mostre nome, categoria e preço dos produtos que são Eletrônicos ou Acessórios, custam menos de 100, do mais barato para o mais caro, só os 3 primeiros."* Resposta: Garrafa térmica (45.00), Mouse sem fio (59.90) e Fone de ouvido (89.90).

6. Resumindo Dados: Contar, Somar e Agrupar

Até aqui você pediu linhas do banco. Agora vai pedir **resumos**: quantos clientes temos? Qual o preço médio? Quanto vende cada categoria? Isso é o dia a dia de quem trabalha com dados.

Funções de agregação

Uma **função** é uma "mini-máquina" que recebe valores e devolve um resultado. As funções de **agregação** pegam várias linhas e devolvem **um único valor**.

Função	O que faz	Exemplo	Resultado
`COUNT(*)`	Conta linhas	`SELECT COUNT(*) FROM produtos;`	8
`COUNT(coluna)`	Conta linhas onde a coluna **não é vazia**	`SELECT COUNT(cidade) FROM clientes;`	4 (o Diego não entra)
`SUM(coluna)`	Soma	`SELECT SUM(preco) FROM produtos;`	575.90
`AVG(coluna)`	Média	`SELECT ROUND(AVG(preco), 2) FROM produtos;`	71.99

Função	O que faz	Exemplo	Resultado
`MIN(coluna)`	Menor valor	`SELECT MIN(preco) FROM produtos;`	2.50
`MAX(coluna)`	Maior valor	`SELECT MAX(preco) FROM produtos;`	149.90

`ROUND(número, 2)` arredonda para 2 casas decimais. Você pode usar várias funções de uma vez e combinar com `WHERE`:

```
SELECT COUNT(*) AS qtd, MIN(preco) AS mais_barato, MAX(preco) AS mais_caro
FROM produtos
WHERE categoria = 'Eletrônicos';
```

qtd	mais_barato	mais_caro
3	59.90	149.90

Para contar valores diferentes, use `COUNT(DISTINCT coluna)`. Exemplo: `SELECT COUNT(DISTINCT cidade) FROM clientes;` devolve 3.

GROUP BY: resumo por grupo

E se você quiser a contagem **de cada categoria**, não só o total? Use o `GROUP BY` ("agrupar por"). Imagine separar todos os produtos em caixas, uma caixa por categoria, e aplicar a contagem em cada caixa.

```
SELECT categoria, COUNT(*) AS qtd, ROUND(AVG(preco), 2) AS preco_medio
FROM produtos
GROUP BY categoria;
```

categoria	qtd	preco_medio
Papelaria	2	10.70
Acessórios	2	87.45
Eletrônicos	3	99.90
Livros	1	79.90

Outro exemplo, clientes por cidade:

```
SELECT cidade, COUNT(*) AS qtd FROM clientes GROUP BY cidade;
```

cidade	qtd
Rio de Janeiro	2
Niterói	1
São Paulo	1
NULL	1

> **Regra:** toda coluna que aparece no `SELECT` e **não** está dentro de uma função de agregação precisa estar no `GROUP BY`. Se faltar, o MySQL 8 devolve um erro (`ONLY_FULL_GROUP_BY`), porque não saberia qual valor mostrar.

HAVING: filtrar os grupos

O `WHERE` filtra **linhas antes** de agrupar. Para filtrar **grupos depois** de contados, use o `HAVING`:

```
SELECT categoria, COUNT(*) AS qtd
FROM produtos
GROUP BY categoria
HAVING COUNT(*) >= 2;
```

Resultado: só Papelaria, Acessórios e Eletrônicos (Livros tem apenas 1 produto e fica de fora).

Quero filtrar...	Use	Exemplo
Linhas, antes de resumir	`WHERE`	`WHERE preco > 50`
Grupos, depois de resumir	`HAVING`	`HAVING COUNT(*) >= 2`

A ordem completa de escrita é: `SELECT`, `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`, `LIMIT`.

Funções que mexem em textos

Função	O que faz	Exemplo	Resultado
`UPPER(t)` / `LOWER(t)`	Maiúsculas / minúsculas	`UPPER('rio')`	`RIO`
`CONCAT(a, b, ...)`	Junta textos	`CONCAT('Olá, ', nome)`	`Olá, Ana Souza`
`CHAR_LENGTH(t)`	Número de caracteres	`CHAR_LENGTH('Ana')`	`3`
`SUBSTRING(t, início, tam)`	Pega um pedaço	`SUBSTRING('banana', 1, 3)`	`ban`
`REPLACE(t, velho, novo)`	Troca trechos	`REPLACE('a-b', '-', '/')`	`a/b`
`TRIM(t)`	Tira espaços das pontas	`TRIM(' oi ')`	`oi`

> **Atenção:** no MySQL, juntar textos é com `CONCAT()`. O símbolo `||` significa "OU", como o `OR`, e não junta textos como em outros bancos.

Funções que mexem em datas

Função	O que faz	Exemplo
`NOW()`	Data e hora de agora	`2026-10-08 14:30:00`
`CURDATE()`	Só a data de hoje	`2026-10-08`
`DATE_FORMAT(d, formato)`	Mostra a data no formato que você quiser	`DATE_FORMAT(NOW(), '%d/%m/%Y')` devolve `08/10/2026`
`DATEDIFF(d1, d2)`	Dias de diferença entre duas datas	`DATEDIFF('2026-12-25', CURDATE())`
`YEAR(d)`, `MONTH(d)`, `DAY(d)`	Pega o ano, o mês ou o dia	`YEAR(NOW())` devolve `2026`
`DATE_ADD(d, INTERVAL 7 DAY)`	Soma tempo a uma data	Daqui a 7 dias

No `DATE_FORMAT`: `%d` é o dia, `%m` o mês, `%Y` o ano com 4 dígitos, `%H` a hora e `%i` os minutos.

CASE: "se isso, então aquilo"

O `CASE` cria uma coluna nova aplicando regras, como um "se... senão":

```
SELECT nome, preco,
  CASE
    WHEN preco < 50  THEN 'Barato'
    WHEN preco < 100 THEN 'Médio'
    ELSE 'Caro'
  END AS faixa
FROM produtos;
```

nome	preco	faixa
Caderno 20 matérias	18.90	Barato
Caneta azul	2.50	Barato
Mochila escolar	129.90	Caro
Fone de ouvido	89.90	Médio
Mouse sem fio	59.90	Médio
Teclado	149.90	Caro
Garrafa térmica	45.00	Barato
Livro de SQL	79.90	Médio

O MySQL testa as condições de cima para baixo e usa a **primeira** que for verdadeira.

7. Mais de uma Tabela: Relacionamentos e JOIN

Aqui está o motivo de o nome ser **banco de dados relacional**: as tabelas se **relacionam** entre si. É o conceito que mais assusta iniciantes e o que mais traz poder. Vamos com calma.

Por que dividir em várias tabelas?

Imagine guardar os pedidos de uma loja em uma única planilha, escrevendo o nome, o e-mail e a cidade do cliente em **cada** pedido:

pedido	cliente	e-mail	cidade	produto
1	Ana Souza	ana@email.com	Rio de Janeiro	Fone de ouvido
2	Ana Souza	ana@email.com	Rio de Janeiro	Mochila escolar

A Ana aparece repetida. Se ela mudar de e-mail, você teria de corrigir em todas as linhas, e é fácil esquecer uma. A solução é guardar o cliente **uma vez só** na tabela `clientes` e, no pedido, anotar apenas o **número** (`id`) dele. Esse número que aponta para outra tabela se chama **chave estrangeira**.

Termo	Explicação	Analogia
Chave primária (PK)	O número único de uma linha na **sua própria** tabela	Seu CPF
Chave estrangeira (FK)	Coluna que guarda a chave primária de **outra** tabela, criando o vínculo	Seu CPF escrito em um formulário de matrícula

Criando as tabelas de pedidos

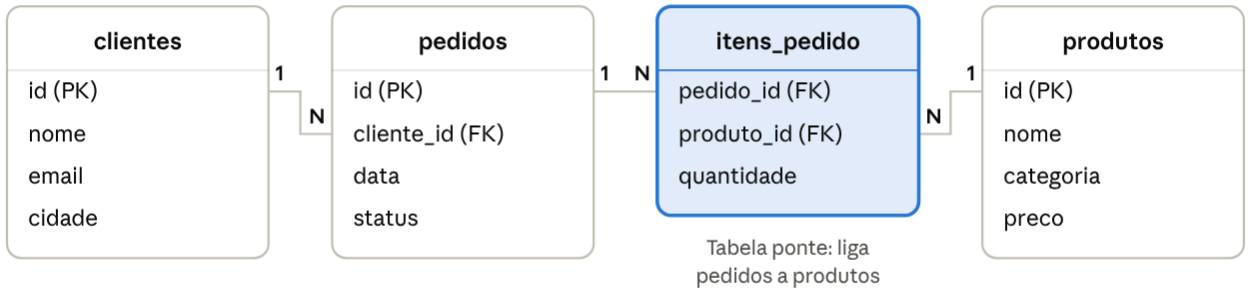
```
CREATE TABLE pedidos (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  cliente_id INT NOT NULL,
  data       DATE NOT NULL,
  status     VARCHAR(20) NOT NULL,
  FOREIGN KEY (cliente_id) REFERENCES clientes(id)
);

CREATE TABLE itens_pedido (
  pedido_id  INT NOT NULL,
  produto_id INT NOT NULL,
  quantidade INT NOT NULL,
  PRIMARY KEY (pedido_id, produto_id),
  FOREIGN KEY (pedido_id)  REFERENCES pedidos(id),
  FOREIGN KEY (produto_id) REFERENCES produtos(id)
);
```

- `FOREIGN KEY (cliente_id) REFERENCES clientes(id)` diz: "o valor de `cliente_id` precisa existir na coluna `id` de `clientes`". O MySQL passa a **recusar** um pedido de um cliente que não existe, e também recusa apagar um cliente que ainda tem pedidos.

- A tabela `itens_pedido` é uma **"ponte"**: um pedido tem vários produtos e um produto aparece em vários pedidos. Cada linha liga um pedido a um produto e diz a quantidade. Sua chave primária é a **combinação** de pedido e produto.

Um cliente tem vários pedidos; itens_pedido liga pedidos a produtos



PK = chave primária · FK = chave estrangeira · 1 = um · N = vários

mapa do banco loja · 4 tabelas

O número 1 marca o lado "um" e o N o lado "vários": um cliente tem vários pedidos, e um pedido tem vários itens. A tabela em destaque faz a ponte entre pedidos e produtos.

Vamos colocar dados de exemplo:

```
INSERT INTO pedidos (cliente_id, data, status) VALUES
  (1, '2026-09-10', 'pago'),
  (1, '2026-09-25', 'pago'),
  (2, '2026-09-28', 'pago'),
  (3, '2026-10-02', 'cancelado'),
  (5, '2026-10-05', 'aberto');

INSERT INTO itens_pedido (pedido_id, produto_id, quantidade) VALUES
  (1, 4, 1), (1, 1, 2),
  (2, 3, 1),
  (3, 6, 1), (3, 5, 1), (3, 2, 5),
  (4, 8, 1),
  (5, 7, 2);
```

JOIN: juntando as informações

Os dados estão separados. O `JOIN` ("juntar") **cola as tabelas de volta, na hora da consulta**, ligando as linhas que se correspondem.

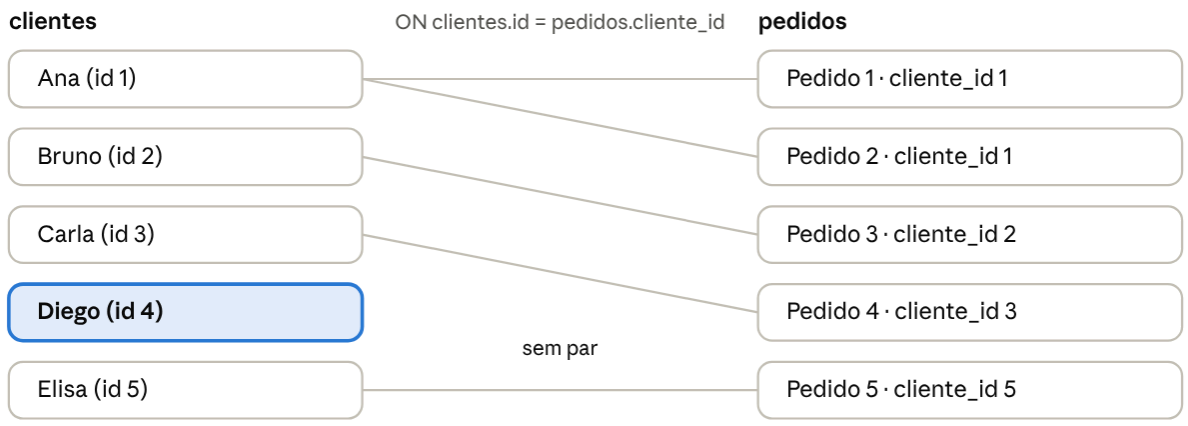
```
SELECT p.id AS pedido, c.nome, p.data, p.status
FROM pedidos p
INNER JOIN clientes c ON c.id = p.cliente_id;
```

pedido	nome	data	status
1	Ana Souza	2026-09-10	pago
2	Ana Souza	2026-09-25	pago
3	Bruno Lima	2026-09-28	pago
4	Carla Dias	2026-10-02	cancelado
5	Elisa Nunes	2026-10-05	aberto

Decifrando:

- `FROM pedidos p`: a tabela principal. O `p` é um **apelido** (alias) para não repetir o nome inteiro.
- `INNER JOIN clientes c`: junte com a tabela `clientes`, apelidada `c`.
- `ON c.id = p.cliente_id`: **a regra de ligação**: "o `id` do cliente é igual ao `cliente_id` do pedido". Sem o `ON` o resultado vira uma combinação sem sentido de todas as linhas com todas.
- `c.nome` e `p.data`: o prefixo diz **de qual tabela** vem a coluna. É obrigatório quando o nome existe nas duas (como `id`).

O JOIN liga cada cliente aos seus pedidos, e o Diego fica sem par



INNER JOIN: o Diego some do resultado.
LEFT JOIN: o Diego continua, com os dados de pedido vazios (NULL).

JOIN · 5 clientes, 5 pedidos

Cada linha liga um cliente ao pedido dele. Sem linha, não há par: é por isso que o tipo de JOIN escolhido decide se o Diego aparece ou não.

Os tipos principais de JOIN

Tipo	Traz	Analogia
`INNER JOIN`	Só as linhas que têm par **nas duas** tabelas	Só quem aparece nas duas listas de convidados
`LEFT JOIN`	**Todas** as da tabela da esquerda, mesmo sem par (as colunas da direita vêm `NULL`)	Todos os convidados da primeira lista, com ou sem acompanhante
`RIGHT JOIN`	O inverso do `LEFT`: todas as da direita	Raramente usado; dá para trocar a ordem das tabelas e usar `LEFT`

O Diego nunca fez pedido. Com `INNER JOIN` ele some do resultado. Com `LEFT JOIN` ele continua lá:

```
SELECT c.nome, COUNT(p.id) AS pedidos
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
GROUP BY c.id, c.nome;
```

nome	pedidos
Ana Souza	2
Bruno Lima	1
Carla Dias	1
Diego Rocha	0
Elisa Nunes	1

Um truque clássico: **quais clientes nunca compraram?** Fazemos o `LEFT JOIN` e ficamos com quem não achou par:

```
SELECT c.nome
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
WHERE p.id IS NULL;
```

Resultado: Diego Rocha.

> **Observação:** o MySQL não tem o `FULL JOIN` ("todas de ambos os lados") que existe em outros bancos. Raramente faz falta, e há um jeito de simular quando necessário.

Juntando várias tabelas de uma vez

Quanto cada cliente gastou em pedidos **pagos**? Precisamos de quatro tabelas encadeadas: cliente, pedido, itens e produto (para saber o preço).

```
SELECT c.nome, SUM(i.quantidade * pr.preco) AS total_gasto
FROM clientes c
JOIN pedidos p       ON p.cliente_id = c.id
JOIN itens_pedido i  ON i.pedido_id  = p.id
JOIN produtos pr     ON pr.id        = i.produto_id
WHERE p.status = 'pago'
GROUP BY c.id, c.nome
ORDER BY total_gasto DESC;
```

nome	total_gasto
Ana Souza	257.60
Bruno Lima	222.30

É só repetir o padrão `JOIN tabela ON regra_de_ligação`. Escrever só `JOIN` equivale a `INNER JOIN`. Leia sempre da esquerda para a direita, seguindo o caminho: cliente, pedido, item, produto.

> **Armadilha comum:** em um `LEFT JOIN`, se você colocar no `WHERE` uma condição sobre a tabela da **direita** (por exemplo `WHERE p.status = 'pago'`), as linhas sem par somem e o `LEFT JOIN` vira, na prática, um `INNER JOIN`. Para manter os "sem par", coloque a condição no `ON`: `LEFT JOIN pedidos p ON p.cliente_id = c.id AND p.status = 'pago'`.

8. Consultas Dentro de Consultas

Às vezes, para responder uma pergunta, você precisa **primeiro** responder outra. Exemplo: "quais produtos custam mais que a média?" Você precisa saber a média antes de comparar. O SQL permite colocar uma consulta dentro da outra.

Subconsultas

Uma **subconsulta** é um `SELECT` escrito entre parênteses, dentro de outro `SELECT`. O MySQL executa o de dentro primeiro e usa o resultado no de fora.

```
SELECT nome, preco
FROM produtos
WHERE preco > (SELECT AVG(preco) FROM produtos);
```

A média dos preços é 71,99. A consulta de fora então mostra só o que passa disso:

nome	preco
Mochila escolar	129.90
Fone de ouvido	89.90
Teclado	149.90
Livro de SQL	79.90

Subconsulta que devolve uma lista, usada com `IN`: "clientes que fizeram algum pedido pago".

```
SELECT nome FROM clientes
WHERE id IN (SELECT cliente_id FROM pedidos WHERE status = 'pago');
```

Resultado: Ana Souza e Bruno Lima.

Subconsulta com EXISTS, que só pergunta "existe alguma linha assim?". Costuma ser eficiente e é a forma mais segura para o caso contrário (quem **não** tem):

```
SELECT nome FROM clientes c
WHERE NOT EXISTS (SELECT 1 FROM pedidos p WHERE p.cliente_id = c.id);
```

Resultado: Diego Rocha. Esse `1` é só um valor qualquer: o `EXISTS` ignora o conteúdo e olha apenas se há resultado.

> **Por que não `NOT IN`?** Se a lista devolvida pela subconsulta tiver um `NULL` no meio, o `NOT IN` não devolve nada, sem avisar. O `NOT EXISTS` não tem esse problema.

Subconsulta no lugar de uma tabela: o resultado de uma consulta pode ser tratado como se fosse uma tabela. Nesse caso, o MySQL exige que você dê um **apelido** a ela:

```
SELECT t.categoria, t.media
FROM (
  SELECT categoria, ROUND(AVG(preco), 2) AS media
  FROM produtos
  GROUP BY categoria
) AS t
WHERE t.media > 80;
```

categoria	media
Acessórios	87.45
Eletrônicos	99.90

CTE: dando nome a uma etapa

Subconsultas aninhadas ficam difíceis de ler. A **CTE** (*Common Table Expression*, "tabela temporária nomeada") resolve: você dá um nome a cada etapa e depois usa esse nome, como se montasse a resposta por partes. Começa com `WITH`.

Pergunta: **quais pedidos passaram de R$ 100, e de quem são?**

```
WITH total_pedido AS (
  SELECT i.pedido_id, SUM(i.quantidade * pr.preco) AS total
  FROM itens_pedido i
  JOIN produtos pr ON pr.id = i.produto_id
  GROUP BY i.pedido_id
)
SELECT p.id AS pedido, c.nome, t.total
FROM total_pedido t
JOIN pedidos  p ON p.id = t.pedido_id
JOIN clientes c ON c.id = p.cliente_id
WHERE t.total > 100
ORDER BY t.total DESC;
```

pedido	nome	total
3	Bruno Lima	222.30
2	Ana Souza	129.90
1	Ana Souza	127.70

Leia assim: *"Primeiro, calcule o total de cada pedido e chame isso de `total_pedido`. Depois, junte com pedidos e clientes e mostre só os acima de 100."*

> **Versão do MySQL:** CTEs (`WITH`) e as *window functions* da próxima seção exigem o **MySQL 8.0 ou mais novo**. Confira a sua com `SELECT VERSION();`. Se for 5.7 ou anterior, use subconsultas e considere atualizar.

UNION: empilhar resultados

Enquanto o `JOIN` coloca tabelas **lado a lado**, o `UNION` coloca resultados **um embaixo do outro**. As consultas precisam ter o mesmo número de colunas, com tipos parecidos.

```
SELECT 'Cliente' AS tipo, nome FROM clientes
UNION ALL
SELECT 'Produto', nome FROM produtos;
```

O `UNION` sozinho remove linhas repetidas; o `UNION ALL` mantém tudo (e é mais rápido).

Qual escolher?

Situação	Ferramenta
Comparar com um valor calculado (a média, o maior preço)	Subconsulta entre parênteses no `WHERE`
Checar se existe (ou não existe) algo relacionado	`EXISTS` / `NOT EXISTS`
Trazer colunas de outra tabela	`JOIN`
Consulta longa, em várias etapas	CTE (`WITH`)
Somar listas de resultados parecidos	`UNION` / `UNION ALL`

9. Recursos Avançados, com Calma

Se você chegou até aqui, já sabe o essencial. Esta seção apresenta quatro ideias que separam quem "sabe consultar" de quem "sabe trabalhar com banco de dados de verdade": **índices**, **transações**, **rankings** e **views**. Não precisa decorar tudo: o objetivo é você saber que existem e para que servem.

Índices: o "índice remissivo" do banco

Quando você quer achar "Ana" em um livro de 500 páginas, não lê página por página: vai ao **índice remissivo** no final e pula direto para a página certa. O **índice** de um banco de dados faz exatamente isso.

Sem índice, para achar os pedidos do cliente 1, o MySQL lê **a tabela inteira**, linha por linha. Com milhões de linhas, isso demora. Com um índice, ele vai direto ao ponto.

```
CREATE INDEX idx_pedidos_data ON pedidos (data);
SHOW INDEX FROM pedidos;                  -- lista os índices da tabela
DROP INDEX idx_pedidos_data ON pedidos;   -- remove um índice
```

Pergunta	Resposta
O que já tem índice automático?	A chave primária, colunas `UNIQUE` e chaves estrangeiras
Onde vale criar índice?	Em colunas muito usadas em `WHERE`, em `JOIN` e em `ORDER BY`
Qual o preço?	Ocupa espaço e deixa `INSERT`, `UPDATE` e `DELETE` um pouco mais lentos. Por isso não se indexa tudo
Preciso em tabelas pequenas?	Não. A diferença só aparece com muitos dados

Para descobrir se uma consulta está lenta por falta de índice, coloque `EXPLAIN` na frente dela. Ele **não executa** a consulta: mostra o "plano" que o MySQL usaria.

```
EXPLAIN SELECT * FROM pedidos WHERE data = '2026-09-10';
```

Na coluna `type` do resultado, o valor **`ALL`** significa "leu a tabela inteira" (mau sinal em tabela grande), e a coluna `key` mostra qual índice foi usado (vazio = nenhum).

> **Pegadinha:** o índice some do jogo se você aplicar uma função na coluna. `WHERE YEAR(data) = 2026` não usa o índice de `data`; já `WHERE data >= '2026-01-01' AND data < '2027-01-01'` usa. O mesmo vale para `LIKE '%texto'` (começando com `%`).

Transações: "tudo ou nada"

Pense em uma transferência bancária de R$ 100 da conta A para a conta B. São **dois passos**: tirar de A e colocar em B. E se a luz acabar entre os dois? O dinheiro sumiria. Uma **transação** agrupa os passos para que **ou todos aconteçam, ou nenhum**.

```
START TRANSACTION;                                    -- começa o bloco
UPDATE contas SET saldo = saldo - 100 WHERE id = 1;  -- tira de A
UPDATE contas SET saldo = saldo + 100 WHERE id = 2;  -- põe em B
COMMIT;                                               -- confirma tudo
```

Se algo deu errado no meio, em vez de `COMMIT` você usa `ROLLBACK`, que **desfaz tudo** desde o `START TRANSACTION`, como se nada tivesse acontecido.

Comando	Efeito
`START TRANSACTION`	Abre o bloco. Nada é definitivo ainda
`COMMIT`	Confirma e torna tudo permanente
`ROLLBACK`	Cancela e volta ao estado de antes

Por padrão o MySQL funciona em modo **autocommit**: cada comando solto já é confirmado na hora. É por isso que um `DELETE` errado não tem volta. Dentro de uma transação, você ganha uma "rede de segurança".

As transações seguem quatro garantias, conhecidas pela sigla **ACID**:

Letra	Garantia	Em português simples
A	Atomicidade	Tudo ou nada
C	Consistência	O banco nunca fica em um estado que quebre as regras
I	Isolamento	Duas pessoas mexendo ao mesmo tempo não se atrapalham
D	Durabilidade	Depois do `COMMIT`, os dados sobrevivem até a uma queda de energia

> **Aqui entra o InnoDB, lembra?** É o motor de armazenamento que oferece transações. Como ele é o padrão, em tabelas criadas normalmente tudo isso já funciona.

Window functions: rankings sem perder as linhas

O `GROUP BY` resume várias linhas em uma. Às vezes você quer o contrário: **manter todas as linhas** e acrescentar um cálculo ao lado, como a posição de cada produto dentro da sua categoria. Para isso existem as *window functions* ("funções de janela"), do MySQL 8 em diante.

```
SELECT categoria, nome, preco,
  RANK() OVER (PARTITION BY categoria ORDER BY preco DESC) AS posicao
FROM produtos
ORDER BY categoria, posicao;
```

categoria	nome	preco	posicao
Acessórios	Mochila escolar	129.90	1
Acessórios	Garrafa térmica	45.00	2
Eletrônicos	Teclado	149.90	1
Eletrônicos	Fone de ouvido	89.90	2
Eletrônicos	Mouse sem fio	59.90	3
Livros	Livro de SQL	79.90	1
Papelaria	Caderno 20 matérias	18.90	1
Papelaria	Caneta azul	2.50	2

O trecho `OVER (PARTITION BY categoria ORDER BY preco DESC)` significa: *"separe em grupos por categoria (`PARTITION BY`), ordene cada grupo do maior para o menor preço e numere"*. Funções úteis:

Função	O que faz
`ROW_NUMBER()`	Numera 1, 2, 3, 4 sem repetir
`RANK()`	Numera, mas empates dividem a posição (1, 2, 2, 4)
`LAG(coluna)`	Pega o valor da linha **anterior** (útil para comparar com o mês passado)
`SUM(coluna) OVER (ORDER BY ...)`	Soma acumulada

Um uso clássico é pegar o **primeiro de cada grupo**, como o produto mais caro de cada categoria, combinando com uma CTE:

```
WITH ranking AS (
  SELECT categoria, nome, preco,
    ROW_NUMBER() OVER (PARTITION BY categoria ORDER BY preco DESC) AS pos
  FROM produtos
)
SELECT categoria, nome, preco FROM ranking WHERE pos = 1;
```

categoria	nome	preco
Acessórios	Mochila escolar	129.90
Eletrônicos	Teclado	149.90
Livros	Livro de SQL	79.90
Papelaria	Caderno 20 matérias	18.90

Views: consultas salvas com nome

Uma **view** é uma consulta salva, que você usa depois como se fosse uma tabela. Perfeita para aquela consulta comprida que você repete toda semana.

```
CREATE VIEW vw_total_pedido AS
SELECT i.pedido_id, SUM(i.quantidade * pr.preco) AS total
FROM itens_pedido i
JOIN produtos pr ON pr.id = i.produto_id
GROUP BY i.pedido_id;

SELECT * FROM vw_total_pedido WHERE total > 100;
```

A view **não guarda dados próprios**: toda vez que você a consulta, o MySQL roda o `SELECT` por trás. Também serve para mostrar a alguém só algumas colunas, escondendo as demais.

Para saber que existe

Você não precisa dominar isso agora, mas vai ouvir esses nomes:

Recurso	O que é
Stored procedure	Um "programinha" salvo dentro do banco, executado com `CALL nome()`
Function	Como a procedure, mas devolve um valor e pode ser usada dentro de um `SELECT`
Trigger (gatilho)	Código que roda **sozinho** quando algo acontece, como registrar em um histórico sempre que um preço mudar
Event	Tarefa agendada dentro do banco, tipo "todo dia às 3h, apague pedidos antigos"

Recurso	O que é
JSON	Tipo de coluna para guardar dados com formato flexível, como configurações

10. Segurança e Backup

Dados valem dinheiro e, muitas vezes, são pessoais e sensíveis. Esta seção cobre o essencial para não ter dor de cabeça: **quem pode fazer o quê**, **como evitar um ataque clássico** e **como não perder tudo**.

Usuários e permissões

O MySQL permite criar vários **usuários**, cada um com um conjunto de **permissões**, como crachás de acesso em um prédio: a recepção entra em alguns andares, o diretor em todos.

Se você instalou o MySQL, já existe o usuário `root`, o administrador com poder total. A regra de ouro é o **princípio do menor privilégio**: cada pessoa ou programa recebe **só o que precisa**. Um site de loja, por exemplo, precisa ler e gravar pedidos, mas **nunca** deveria poder apagar o banco inteiro. Por isso, **nunca conecte aplicações usando o `root`**.

```
-- Cria o usuário 'app_loja', que só pode se conectar da própria máquina
CREATE USER 'app_loja'@'localhost' IDENTIFIED BY 'UmaSenhaBemLonga#2026';

-- Dá permissão para ler e gravar dados no banco 'loja' (e nada além disso)
GRANT SELECT, INSERT, UPDATE, DELETE ON loja.* TO 'app_loja'@'localhost';

SHOW GRANTS FOR 'app_loja'@'localhost';        -- confere as permissões
REVOKE DELETE ON loja.* FROM 'app_loja'@'localhost';   -- retira uma
permissão
DROP USER 'app_loja'@'localhost';              -- remove o usuário
```

Trecho	Significado
`'app_loja'@'localhost'`	Nome do usuário e **de onde** ele pode se conectar. `localhost` = só do próprio computador
`GRANT`	"Conceder" permissões
`REVOKE`	"Retirar" permissões
`loja.*`	Todas as tabelas (`*`) do banco `loja`

Uma **senha forte** é longa (12 caracteres ou mais), mistura tipos de caracteres e não é usada em nenhum outro lugar. Um gerenciador de senhas ajuda.

SQL Injection: o ataque mais famoso

Imagine uma página de login. O site pega o que você digita e monta um comando SQL **juntando textos**:

```
SELECT * FROM usuarios WHERE login = 'ana' AND senha = '1234';
```

Agora um atacante, em vez de um login, digita no campo: `' OR '1'='1`. O comando montado vira:

```
SELECT * FROM usuarios WHERE login = '' OR '1'='1' AND senha = '';
```

Como `'1'='1'` é sempre verdadeiro, o banco devolve usuários sem exigir senha. O atacante **injetou SQL** no seu comando. Variações do ataque conseguem ler, alterar ou apagar o banco inteiro.

A defesa é **nunca montar comandos juntando texto que veio do usuário**. Em vez disso, use **consultas parametrizadas** (também chamadas *prepared statements*): o comando é enviado separado dos valores, e o MySQL trata o que o usuário digitou **sempre como dado, nunca como comando**.

```
# ERRADO: juntando texto
cursor.execute("SELECT * FROM usuarios WHERE login = '" + login + "'")

# CERTO: o %s é um espaço reservado, e o valor vai separado
cursor.execute("SELECT * FROM usuarios WHERE login = %s", (login,))
```

Todas as linguagens (PHP, Python, JavaScript, Java) oferecem esse recurso. Frameworks e ORMs usam por padrão, mas vale conferir.

> **Para guardar:** qualquer texto vindo de formulário, URL ou arquivo é **suspeito**. Parametrize sempre.

Outros cuidados importantes

- **Senhas de usuários do seu sistema** nunca são guardadas em texto puro. Use um "hash" com sal (bcrypt ou argon2), que é uma transformação de mão única.
- **Não deixe o banco aberto na internet.** A porta padrão do MySQL é a `3306`; restrinja o acesso por firewall ou VPN.
- **Mantenha o MySQL atualizado**, pois atualizações corrigem falhas de segurança.

- **Dados pessoais** (CPF, endereço, saúde) estão sujeitos à **LGPD**, a lei brasileira de proteção de dados. Guarde só o necessário e proteja bem.

Backup: a cópia de segurança

Um **backup** é uma cópia dos seus dados guardada em outro lugar. Discos quebram, pessoas apagam coisas sem querer, sites são invadidos. Quem tem backup perde um dia; quem não tem pode perder tudo.

A ferramenta mais comum é o `mysqldump`. Ele gera um arquivo de texto com todos os comandos necessários para **recriar** o banco. Atenção: esses comandos são digitados no **terminal do computador**, e não dentro do Workbench ou do cliente `mysql`.

```
# Fazer o backup do banco 'loja' em um arquivo
mysqldump -u root -p --single-transaction loja > backup_loja.sql

# Restaurar o backup (o banco 'loja' precisa existir, mesmo que vazio)
mysql -u root -p loja < backup_loja.sql
```

O `>` manda o resultado para um arquivo, e o `<` faz o contrário, lê o arquivo e entrega ao MySQL. O `-p` pede a senha.

Boa prática	Por quê
Fazer backups **automáticos**, todo dia	Quem depende de lembrar acaba esquecendo
Guardar a cópia **em outro lugar** (outro disco, nuvem)	Se o computador pegar fogo, a cópia ao lado queima junto
Testar a restauração de vez em quando	Backup que nunca foi restaurado pode estar corrompido sem você saber
Regra **3-2-1**: 3 cópias, em 2 tipos de mídia, 1 delas fora do local	Padrão usado por profissionais

> **Antes de qualquer operação perigosa** (apagar em massa, alterar a estrutura de uma tabela de produção), faça um backup. É o seguro mais barato que existe.

11. Praticando: Exercícios, Erros Comuns e Glossário

SQL se aprende **fazendo**. Tente resolver cada exercício sozinho antes de olhar a resposta. Use o banco `loja` que você construiu. Se algo der errado, leia a mensagem de erro com calma: ela costuma indicar onde está o problema.

Exercícios com respostas

1. Mostre o nome e o e-mail de todos os clientes, em ordem alfabética de nome.

```
SELECT nome, email FROM clientes ORDER BY nome;
```

2. Liste os produtos da categoria Eletrônicos, do mais barato ao mais caro.

```
SELECT nome, preco FROM produtos
WHERE categoria = 'Eletrônicos'
ORDER BY preco ASC;
```

3. Quantos clientes moram no Rio de Janeiro? (Resposta esperada: 2)

```
SELECT COUNT(*) AS total FROM clientes WHERE cidade = 'Rio de Janeiro';
```

4. Quais clientes não informaram a cidade? (Resposta esperada: Diego Rocha)

```
SELECT nome FROM clientes WHERE cidade IS NULL;
```

5. Mostre o preço médio de cada categoria, só das categorias com média acima de 50.

```
SELECT categoria, ROUND(AVG(preco), 2) AS preco_medio
FROM produtos
GROUP BY categoria
HAVING AVG(preco) > 50;
```

Resposta esperada: Acessórios (87.45), Eletrônicos (99.90) e Livros (79.90).

6. Liste o nome do cliente e a data dos pedidos feitos em setembro de 2026. (Esperado: pedidos 1, 2 e 3)

```
SELECT c.nome, p.data
FROM pedidos p
JOIN clientes c ON c.id = p.cliente_id
WHERE p.data BETWEEN '2026-09-01' AND '2026-09-30';
```

7. Quais clientes nunca fizeram pedido? (Resposta esperada: Diego Rocha)

```
SELECT c.nome
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
WHERE p.id IS NULL;
```

8. Calcule o valor total de cada pedido.

```
SELECT i.pedido_id, SUM(i.quantidade * pr.preco) AS total
FROM itens_pedido i
JOIN produtos pr ON pr.id = i.produto_id
GROUP BY i.pedido_id;
```

Resposta esperada: pedido 1 = 127.70; 2 = 129.90; 3 = 222.30; 4 = 79.90; 5 = 90.00.

9. Qual produto foi vendido em maior quantidade? (Resposta esperada: Caneta azul, com 5 unidades)

```
SELECT pr.nome, SUM(i.quantidade) AS qtd
FROM itens_pedido i
JOIN produtos pr ON pr.id = i.produto_id
GROUP BY pr.id, pr.nome
ORDER BY qtd DESC
LIMIT 1;
```

10. Desafio: qual foi o pedido de maior valor de cada cliente?

```
WITH total_pedido AS (
  SELECT i.pedido_id, SUM(i.quantidade * pr.preco) AS total
  FROM itens_pedido i
  JOIN produtos pr ON pr.id = i.produto_id
  GROUP BY i.pedido_id
),
ranking AS (
  SELECT p.cliente_id, t.pedido_id, t.total,
    ROW_NUMBER() OVER (PARTITION BY p.cliente_id ORDER BY t.total DESC) AS
pos
  FROM total_pedido t
  JOIN pedidos p ON p.id = t.pedido_id
)
SELECT c.nome, r.pedido_id, r.total
FROM ranking r
JOIN clientes c ON c.id = r.cliente_id
WHERE r.pos = 1;
```

Esperado: Ana (pedido 2, 129.90), Bruno (pedido 3, 222.30), Carla (pedido 4, 79.90) e Elisa (pedido 5, 90.00).

Erros comuns de iniciante

Erro	O que acontece	Como evitar
Esquecer o `;` no final	O MySQL fica esperando mais texto e nada acontece	Termine todo comando com `;`
Esquecer o `WHERE` em `UPDATE` ou `DELETE`	Altera ou apaga a tabela inteira	Teste antes com `SELECT` e o mesmo `WHERE`
Esquecer de escolher o banco	Erro `No database selected`	Rode `USE loja;`
Comparar com `= NULL`	Nunca acha nada	Use `IS NULL`
Esquecer aspas em textos	Erro de sintaxe ou `Unknown column`	Texto vai entre aspas simples: `'Ana'`
Vírgula sobrando antes do `FROM`	Erro de sintaxe	A última coluna da lista não leva vírgula
`JOIN` sem o `ON`	Combina tudo com tudo, e o resultado explode	Sempre escreva a regra de ligação
Misturar `WHERE` e `HAVING`	Erro ou resultado errado	`WHERE` filtra linhas; `HAVING` filtra grupos
Confiar na ordem sem `ORDER BY`	A ordem pode mudar	Peça a ordem que quer
Colocar senha ou dados reais em lugar público	Vazamento	Use dados de mentira para treinar

Glossário: as palavras que você vai ouvir

Palavra	Significado simples
Banco de dados	Lugar organizado onde as informações ficam guardadas
SGBD	O programa que gerencia o banco (MySQL, PostgreSQL...)
SQL	A linguagem usada para conversar com o banco
MySQL	Um dos programas de banco de dados mais usados do mundo

Palavra	Significado simples
Servidor	O programa MySQL que fica ligado guardando os dados
Cliente	O programa onde você digita comandos (Workbench, terminal)
Consulta / query	Uma pergunta ou comando enviado ao banco
Tabela	Conjunto de dados sobre um assunto, em linhas e colunas
Coluna / campo	Um tipo de informação (nome, preço)
Linha / registro	Uma ocorrência completa (um cliente, um produto)
Chave primária (PK)	Coluna que identifica cada linha de forma única
Chave estrangeira (FK)	Coluna que aponta para a chave primária de outra tabela
Constraint / restrição	Regra que o banco aplica (`NOT NULL`, `UNIQUE`...)
NULL	"Valor desconhecido ou não informado"
JOIN	Juntar dados de tabelas diferentes em uma só consulta
Alias / apelido	Nome curto ou alternativo dado a uma tabela ou coluna
Função de agregação	Resume várias linhas em um valor (`COUNT`, `SUM`, `AVG`)
Subconsulta	Uma consulta dentro de outra
CTE	Etapa nomeada de uma consulta, escrita com `WITH`
Índice	Estrutura que acelera buscas, como o índice de um livro
Transação	Grupo de comandos que acontece por inteiro ou não acontece
COMMIT / ROLLBACK	Confirmar / desfazer uma transação
InnoDB	O "motor de armazenamento" padrão do MySQL, que grava os dados e dá suporte a transações. Você não precisa configurar

Palavra	Significado simples
View	Consulta salva com nome, usada como se fosse uma tabela
root	Usuário administrador, com todos os poderes
SQL injection	Ataque que insere comandos maliciosos por campos de formulário
Backup	Cópia de segurança dos dados
LGPD	Lei brasileira de proteção de dados pessoais

Plano de estudo sugerido

Treinar um pouco **todo dia** vale mais do que estudar muito uma vez por semana. Trinta a quarenta minutos já ajudam.

Semana	Foco	O que fazer
1	Seções 1 a 3	Instalar, criar o banco `loja` e as tabelas. Repetir de cabeça até se sentir à vontade
2	Seções 4 e 5	Inserir, alterar e consultar. Exercícios 1 a 4
3	Seção 6	Agrupamentos e funções. Exercício 5
4	Seção 7	`JOIN`. Exercícios 6 a 9
5	Seção 8	Subconsultas e CTEs. Exercício 10
6	Seções 9 e 10	Índices, transações, backup. Faça e restaure um `mysqldump`
7 em diante	Projeto próprio	Modele o banco de algo que você conhece (biblioteca, academia, delivery) e invente perguntas para ele

Onde praticar mais: SQLBolt e SQLZoo (tutoriais interativos no navegador, em inglês), W3Schools SQL, HackerRank e LeetCode (seção de bancos de dados, para desafios). **Para consultar depois:** a documentação oficial do MySQL, em `dev.mysql.com/doc`.

> **Mensagem final:** ninguém decora SQL. Profissionais pesquisam sintaxe o tempo todo. O que importa é entender **a lógica** (tabelas, filtros, junções, agrupamentos), e isso você já tem. O resto vem com prática.